第一題，Ackermann

因為我只會遞迴版本的，所以只說明遞迴

$$A(m,n) = \begin{cases} n+1 & \text{若m=0} \\ A(m-1,1) & \text{若m>0且n=0} \\ A(m-1, A(m,n-1)) & \text{若m>0且n>0} \end{cases}$$

這是條件，M=0 回傳 n+1

m>0 並且 N=0 呼叫自己並給 A(m-1,1)

m>0 且 n>0 重複呼叫，給 A(m-1,A(m,n-1))

m\n	0	1	2	3	4	n
0	1	2	3	4	5	$n+1$
1	2	3	4	5	6	$n+2$
2	3	5	7	9	11	$2\cdot(n+3)-3$
3	5	13	29	61	125	$2^{(n+3)}-3$
4	13	65533	$2^{65536}-3$	$A(3, 2^{65536}-3)$	$A(3, A(4, 3))$	$\underbrace{2^{2^{\cdot^{\cdot^{2}}}}}-3$（n+3個數字2）
5	65533	$A(4, 65533)$	$A(4, A(5, 1))$	$A(4, A(5, 2))$	$A(4, A(5, 3))$	
6	$A(5, 1)$	$A(5, A(5, 1))$	$A(5, A(6, 1))$	$A(5, A(6, 2))$	$A(5, A(6, 3))$	

只要稍微改變一下數值，很容易超過程式可執行的範圍

```
auto start = chrono::high_resolution_clock::now();
int result = ackermann_recursive(m, n);
auto end = chrono::high_resolution_clock::now();

chrono::duration<double, std::micro> duration = end - start;
```

我們可以使用 high_resolution_clock 類別來獲取當前的時間點。這個時間點會被存儲在 start 變數中，作為計時的開始。

```
int result = ackermann_recursive(m, n);
```

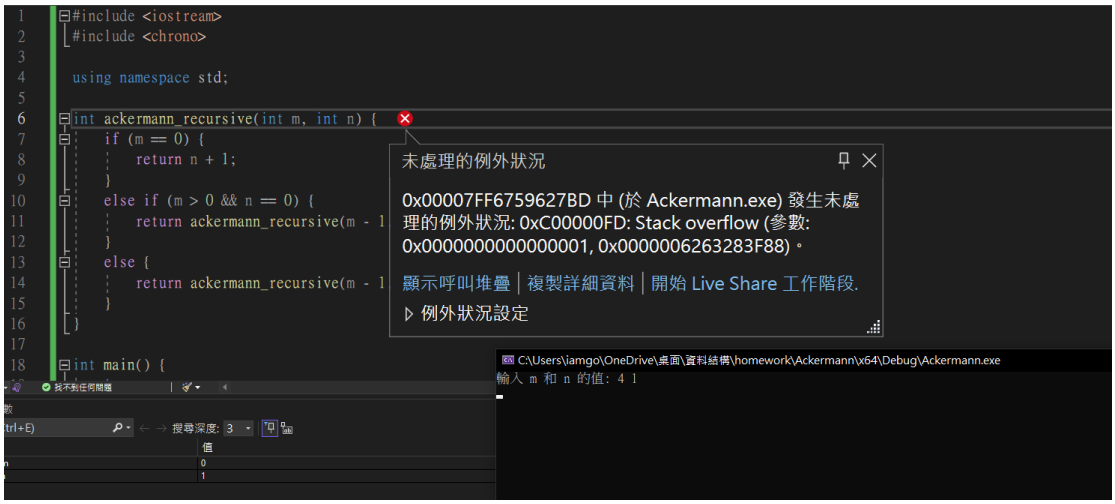
這行執行 Ackermann 函數，並將結果存儲在 result 變數中

```
auto end = chrono::high_resolution_clock::now();
```

這行代碼再次獲取當前的時間點，並將其存儲在 end 變數中，作為計時的結束。

```
chrono::duration<double, std::micro>
```

這行計算了從 start 到 end 的時間差，結果以 duration 物件的形式存儲。

因為運算不多，所以還可以執行

但當輸入 4 1 之後，因為要計算的東西太多，所以崩潰